

GNN Architectures

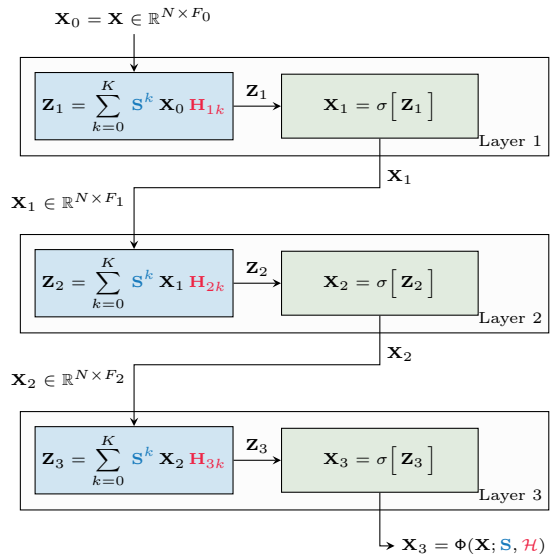
Elvin Isufi

Intelligent Systems Department

May 15, 2026

Graph Convolutional Neural Networks

- ▶ **GCNN**: cascade graph filters and nonlinearities
- ▶ Graph filters capture the data-topology coupling
⇒ The nonlinearity is simply **pointwise**
- ▶ Change the filter → change the GNN



Outline: What is this lecture about?

► How to build graph neural networks?

⇒ Convolutional principle

⇒ Edge varying principle

⇒ Attention principle

⇒ Message passing principle

Convolutional Principle for GNNs

Graph Convolutional Networks

- ▶ GCNNs consider in each layer order K filters: $\mathbf{H}(\mathbf{S}) = \sum_{k=0}^K h_k \mathbf{S}^k$
 $\Rightarrow$ Slower training (?)
- ▶ GCNs consider filters of order $K = 1$ with GSO: $\mathbf{S} = \mathbf{D}^{-\frac{1}{2}} \mathbf{A} \mathbf{D}^{-\frac{1}{2}}$

$$\mathbf{z} = h_0 \mathbf{x} - h_1 \mathbf{S} \mathbf{x}$$

Graph Convolutional Networks

- ▶ GCNNs consider in each layer order K filters: $\mathbf{H}(\mathbf{S}) = \sum_{k=0}^K h_k \mathbf{S}^k$
 $\Rightarrow$ Slower training (?)
- ▶ GCNs consider filters of order $K = 1$ with GSO: $\mathbf{S} = \mathbf{D}^{-\frac{1}{2}} \mathbf{A} \mathbf{D}^{-\frac{1}{2}}$

$$\mathbf{z} = h_0 \mathbf{x} - h_1 \mathbf{S} \mathbf{x}$$

To restrain the number of parameters set $h = h_0 = -h_1$

$$\mathbf{z} = h(\mathbf{I} + \mathbf{D}^{-\frac{1}{2}} \mathbf{A} \mathbf{D}^{-\frac{1}{2}}) \mathbf{x}$$

Graph Convolutional Networks

- ▶ GCNNs consider in each layer order K filters: $\mathbf{H}(\mathbf{S}) = \sum_{k=0}^K h_k \mathbf{S}^k$
 $\Rightarrow$ Slower training (?)
- ▶ GCNs consider filters of order $K = 1$ with GSO: $\mathbf{S} = \mathbf{D}^{-\frac{1}{2}} \mathbf{A} \mathbf{D}^{-\frac{1}{2}}$

$$\mathbf{z} = h_0 \mathbf{x} - h_1 \mathbf{S} \mathbf{x}$$

To restrain the number of parameters set $h = h_0 = -h_1$

$$\mathbf{z} = h(\mathbf{I} + \mathbf{D}^{-\frac{1}{2}} \mathbf{A} \mathbf{D}^{-\frac{1}{2}}) \mathbf{x}$$

- ▶ For multi-features, we have the GCN perceptron

$$\sigma(\mathbf{z}) = \sigma(\mathbf{S} \mathbf{X} \mathbf{H}) := \sigma((\mathbf{I} + \mathbf{D}^{-\frac{1}{2}} \mathbf{A} \mathbf{D}^{-\frac{1}{2}}) \mathbf{X} \mathbf{H})$$

$$\Rightarrow \mathbf{S} = \mathbf{I} + \mathbf{D}^{-\frac{1}{2}} \mathbf{A} \mathbf{D}^{-\frac{1}{2}}$$

Graph Convolutional Networks

- ▶ Using $\mathbf{S} = \mathbf{I} + \mathbf{D}^{-\frac{1}{2}} \mathbf{A} \mathbf{D}^{-\frac{1}{2}}$ has shown empirically numerical instabilities
- ▶ To address this problem, GCN applies a normalization trick

$$\mathbf{S} = \tilde{\mathbf{D}}^{-\frac{1}{2}} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-\frac{1}{2}}$$

$\Rightarrow$ with $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$ and $\tilde{\mathbf{D}}_{ii} = \sum_j \tilde{\mathbf{A}}_{ij}$

Graph Convolutional Networks

- ▶ Using $\mathbf{S} = \mathbf{I} + \mathbf{D}^{-\frac{1}{2}} \mathbf{A} \mathbf{D}^{-\frac{1}{2}}$ has shown empirically numerical instabilities
- ▶ To address this problem, GCN applies a normalization trick

$$\mathbf{S} = \tilde{\mathbf{D}}^{-\frac{1}{2}} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-\frac{1}{2}}$$

⇒ with $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$ and $\tilde{\mathbf{D}}_{ii} = \sum_j \tilde{\mathbf{A}}_{ij}$

- ▶ A **three-layer** GCN is given by

$$\mathbf{Z}_{\text{GCN}} = \sigma(\mathbf{S} \sigma(\mathbf{S} \sigma(\mathbf{S} \mathbf{X} \mathbf{H}_0) \mathbf{H}_1) \mathbf{H}_2)$$

- ⇒ GCNs aggregate **at each layer** information from **one hop neighborhood**
- ⇒ **To cover the whole graph:** the number of layers to match the graph diameter
- ⇒ Can be **limiting** to transform features within each layer

Graph Convolutional Networks

- ▶ Using $\mathbf{S} = \mathbf{I} + \mathbf{D}^{-\frac{1}{2}} \mathbf{A} \mathbf{D}^{-\frac{1}{2}}$ has shown empirically numerical instabilities
- ▶ To address this problem, GCN applies a normalization trick

$$\mathbf{S} = \tilde{\mathbf{D}}^{-\frac{1}{2}} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-\frac{1}{2}}$$

⇒ with $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$ and $\tilde{\mathbf{D}}_{ii} = \sum_j \tilde{\mathbf{A}}_{ij}$

- ▶ A **three-layer** GCN is given by

$$\mathbf{Z}_{\text{GCN}} = \sigma(\mathbf{S} \sigma(\mathbf{S} \sigma(\mathbf{S} \mathbf{X} \mathbf{H}_0) \mathbf{H}_1) \mathbf{H}_2)$$

- ⇒ GCNs aggregate **at each layer** information from **one hop neighborhood**
- ⇒ **To cover the whole graph**: the number of layers to match the graph diameter
- ⇒ Can be **limiting** to transform features within each layer

- ▶ **Q**: When will it work then?

Why do GCNNs Work?

Permutation Equivariance

Permutation Equivariance

- **Regular CNNs** work on images because they are equivariant to translations

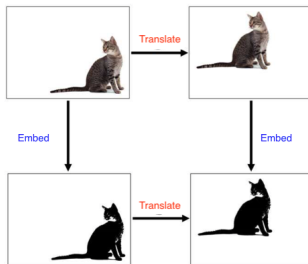


Image credit: Bernhard Kainz

- CNNs can learn from augmented translated inputs
 - ⇒ **Learn** from the cat on the **left** to **classify** the cat on the **right**
 - ⇒ This is a data symmetry that CNNs use
 - ⇒ **What about GCNNs?**

Permutation Equivariance

- ▶ On graphs, we set an arbitrary label of the nodes
 - ⇒ Q: But we can change it and the GCNN should still work, right?

Permutation Equivariance

- ▶ On graphs, we set an arbitrary label of the nodes
⇒ Q: But we can change it and the GCNN should still work, right?

- ▶ Consider a permutation matrix $\mathbf{P}$

$$\{\mathbf{P} \in \{0, 1\}^{N \times N} : \mathbf{P}\mathbf{1} = \mathbf{1}, \mathbf{P}^\top \mathbf{1} = \mathbf{1}\}$$

⇒ Signal permutations $\hat{\mathbf{x}} = \mathbf{P}^\top \mathbf{x}$ reorders the entries of the vector

⇒ Matrix permutations $\hat{\mathbf{S}} = \mathbf{P}^\top \mathbf{S} \mathbf{P}$ reorders the entries of the matrix

Permutation Equivariance

- ▶ On graphs, we set an arbitrary label of the nodes
⇒ Q: But we can change it and the GCNN should still work, right?

- ▶ Consider a permutation matrix $\mathbf{P}$

$$\{\mathbf{P} \in \{0, 1\}^{N \times N} : \mathbf{P}\mathbf{1} = \mathbf{1}, \mathbf{P}^\top \mathbf{1} = \mathbf{1}\}$$

⇒ Signal permutations $\hat{\mathbf{x}} = \mathbf{P}^\top \mathbf{x}$ reorders the entries of the vector

⇒ Matrix permutations $\hat{\mathbf{S}} = \mathbf{P}^\top \mathbf{S} \mathbf{P}$ reorders the entries of the matrix

- ▶ Shifting a permuted signal implies

$$\hat{\mathbf{S}}\hat{\mathbf{x}} = (\mathbf{P}^\top \mathbf{S} \mathbf{P})(\mathbf{P}^\top \mathbf{x}) = \mathbf{P}^\top (\mathbf{S} \mathbf{x}) \quad \mathbf{P} \mathbf{P}^\top = \mathbf{I}$$

equals permuting the shifted signal

⇒ Independent on the arbitrary labeling

Permutation Equivariance

- ▶ Consider the graph convolutional filter $\mathbf{H}(\mathbf{S})\mathbf{x} = \sum_{k=0}^K h_k \mathbf{S}^k \mathbf{x}$
- ▶ Depends on filter parameters $\mathbf{h} = \{h_k\}_k$ and shift operator $\mathbf{S}$; applied to the input signal $\mathbf{x}$

Permutation Equivariance

- ▶ Consider the graph convolutional filter $\mathbf{H}(\mathbf{S})\mathbf{x} = \sum_{k=0}^K h_k \mathbf{S}^k \mathbf{x}$
- ▶ Depends on filter parameters $\mathbf{h} = \{h_k\}_k$ and shift operator $\mathbf{S}$; applied to the input signal $\mathbf{x}$

Proposition

Graph convolutions are *equivariant to permutations*.

Permuted shift: $\hat{\mathbf{S}} = \mathbf{P}^\top \mathbf{S} \mathbf{P}$

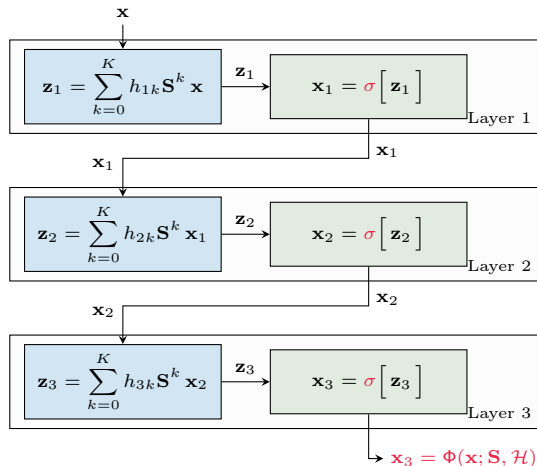
Permuted signals $\hat{\mathbf{x}} = \mathbf{P}^\top \mathbf{x}$

$$\mathbf{H}(\hat{\mathbf{S}})\hat{\mathbf{x}} = \mathbf{P}^\top \mathbf{H}(\mathbf{S})\mathbf{x}$$

$$\text{Proof} \Rightarrow \mathbf{H}(\hat{\mathbf{S}})\hat{\mathbf{x}} = \sum_{k=0}^{\infty} h_k \hat{\mathbf{S}}^k \hat{\mathbf{x}} = \sum_{k=0}^{\infty} h_k (\mathbf{P}^\top \mathbf{S} \mathbf{P})^k \mathbf{P}^\top \mathbf{x} = \mathbf{P}^\top \left(\sum_{k=0}^{\infty} h_k \mathbf{S}^k \mathbf{x} \right) = \mathbf{P}^\top \mathbf{H}(\mathbf{S})\mathbf{x}$$

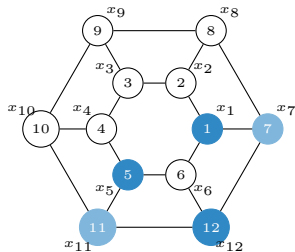
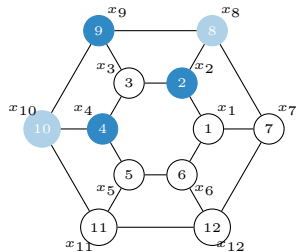
Permutation Equivariance

- ▶ GCNN is a **cascade of layers**
 - ⇒ Graph filters and **pointwise nonlinearities**
- ▶ A pointwise operation does not mix node values
 - ⇒ Independent of the graph
- ▶ **GCNN retains permutation equivariance**



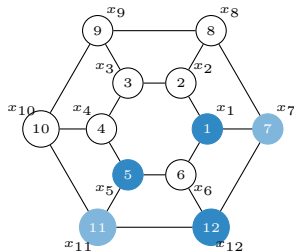
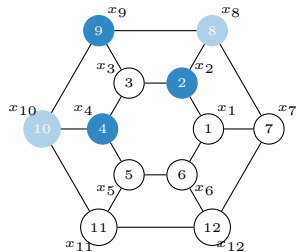
Permutation Equivariance

- Invariance to node relabelings allows GCNNs to **exploit internal symmetries of graph signals**



Permutation Equivariance

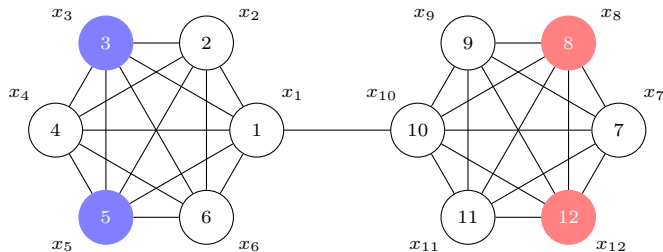
- Invariance to node relabelings allows GCNNs to **exploit internal symmetries of graph signals**



- Although different, both signals are **permutations of one other**
 - ⇒ Permutation equivariance ⇒ The **GCNN** can learn about the right one from seeing the left one
 - ⇒ Permutation equivariance ⇒ makes GCNNs **inductive**

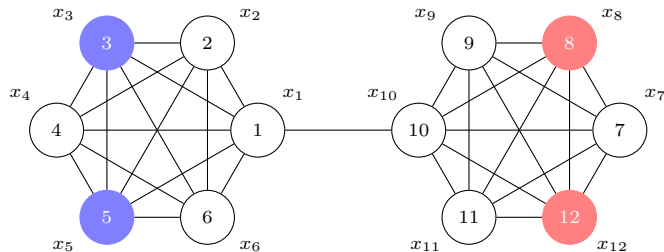
Edge Varying Principle

When Permutation Equivariance is Useful?



- Permutation equivariance is good for some tasks (e.g., rating prediction) but not always
- **RecSys**: user 3, 5, 8 and 12 are interchangeable for rating prediction
⇒ Per. Equiv. needed, as the learning for any of them shall be generalized to the rest

When Permutation Equivariance is Useful?



- ▶ Permutation equivariance is good for some tasks (e.g., rating prediction) but not always
- ▶ **RecSys**: user 3, 5, 8 and 12 are interchangeable for rating prediction
⇒ Per. Equiv. needed, as the learning for any of them shall be generalized to the rest
- ▶ **Node classification**: 3 and 5 are not interchangeable with 8 and 12
⇒ Per. Equiv. not needed, as they belong to different classes

GCNN: Parameter Sharing

Q: How to build GNNs that are not equivariant? (only transductive)

- ▶ We want nodes to **discriminate** information coming from their neighbors
- ▶ In GCNN, each input $\mathbf{x}$ is processed by the filter

$$\mathbf{z} = \mathbf{H}(\mathbf{S})\mathbf{x} = \sum_{k=0}^K h_k \mathbf{S}^k \mathbf{x}$$

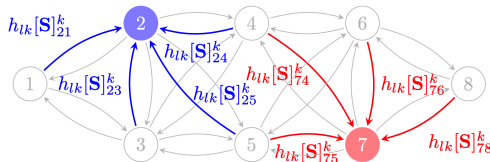
GCNN: Parameter Sharing

Q: How to build GNNs that are not equivariant? (only transductive)

- ▶ We want nodes to **discriminate** information coming from their neighbors
- ▶ In GCNN, each input $\mathbf{x}$ is processed by the filter

$$\mathbf{z} = \mathbf{H}(\mathbf{S})\mathbf{x} = \sum_{k=0}^K h_k \mathbf{S}^k \mathbf{x}$$

$\Rightarrow$ At node i , we have $\mathbf{z}_i = \sum_{k=0}^K h_k \sum_{j \in \mathcal{N}_i \cup \{i\}} \mathbf{S}_{ij}^k \mathbf{x}_j$



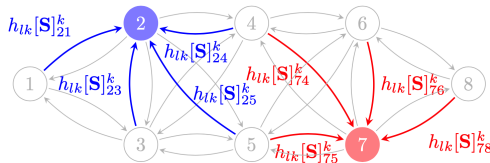
GCNN: Parameter Sharing

Q: How to build GNNs that are not equivariant? (only transductive)

- ▶ We want nodes to **discriminate** information coming from their neighbors
- ▶ In GCNN, each input $\mathbf{x}$ is processed by the filter

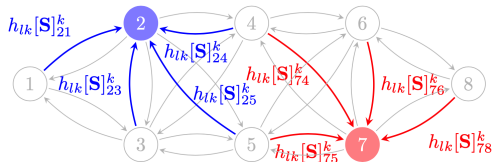
$$\mathbf{z} = \mathbf{H}(\mathbf{S})\mathbf{x} = \sum_{k=0}^K h_k \mathbf{S}^k \mathbf{x}$$

$\Rightarrow$ At node i , we have $\mathbf{z}_i = \sum_{k=0}^K h_k \sum_{j \in \mathcal{N}_i \cup \{i\}} \mathbf{S}_{ij}^k \mathbf{x}_j$



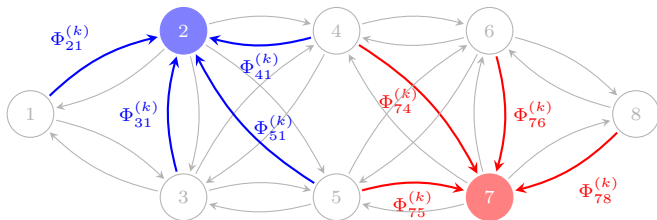
- ▶ Nodes **share** parameters h_k for each shift k
 - $\Rightarrow$ All nodes use the **same parameter** h_k regardless of being different and having different neighbors

Properties of Parameter Sharing



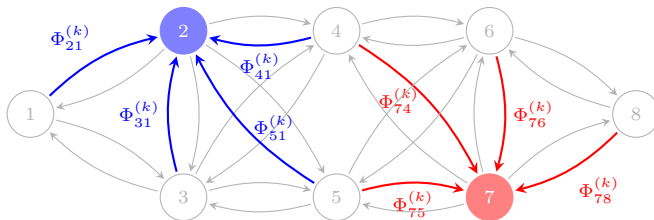
- ▶ Parameter sharing is **advantageous** because
 - ⇒ **Limits** the number of trainable parameters
 - ⇒ **Allows** permutation equivariance
- ▶ However, it is **disadvantageous** because
 - ⇒ **Limits** the discriminatory power
- ▶ We can allocate different weights per node and neighbor

Edge Varying Parameters



- EdgeNets: $\Rightarrow$ Allocate parameters per edges

Edge Varying Parameters



- ▶ EdgeNets: $\Rightarrow$ Allocate parameters per edges
- ▶ More degrees of freedom
 - $\Rightarrow$ Different parameters per **shift**, **node** and **edge**
 - $\Rightarrow$ The signal is shifted by an edge varying **parameter** matrix $\Phi^{(k)}$

$$x_i^{(k)} = \sum_{j \in \mathcal{N}_i \cup i} \Phi_{ij}^{(k)} x_j^{(k-1)}$$

- ▶ **In essence:** make $\mathbf{S}^k \mathbf{x}$ parametric for each k

Edge Varying Filter

- ▶ The edge varying weighting matrices $\Phi^{(k)}$ share the support with $\mathbf{S} + \mathbf{I}$
- ▶ Shifted signal $\mathbf{x}^{(k)}$ can be obtained by the recursion

$$x_i^{(k)} = \sum_{j \in \mathcal{N}_i \cup i} \Phi_{ij}^{(k)} x_j^{(k-1)} \rightarrow \mathbf{x}^{(k)} = \Phi^{(k)} \mathbf{x}^{(k-1)}$$

Edge Varying Filter

- ▶ The edge varying weighting matrices $\Phi^{(k)}$ share the support with $\mathbf{S} + \mathbf{I}$
- ▶ Shifted signal $\mathbf{x}^{(k)}$ can be obtained by the recursion

$$x_i^{(k)} = \sum_{j \in \mathcal{N}_i \cup i} \Phi_{ij}^{(k)} x_j^{(k-1)} \rightarrow \mathbf{x}^{(k)} = \Phi^{(k)} \mathbf{x}^{(k-1)}$$

- ▶ Signal $\mathbf{x}^{(k)}$ can be written as

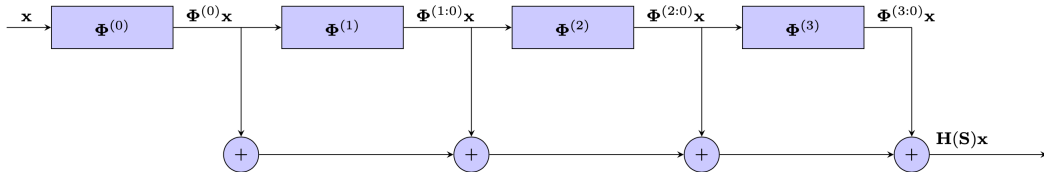
$$\mathbf{x}^{(k)} = \Phi^{(k)} \Phi^{(k-1)} \dots \Phi^{(0)} \mathbf{x} = \Phi^{(k:0)} \mathbf{x}$$

$\Rightarrow$ Learn different graph edge weights $\Phi^{(k)}$ per shift k

Edge Varying Filter (cont.)

- Edge varying graph filter is sum of different shifts $\Phi^{(k:0)}\mathbf{x}$,

$$\mathbf{H}(\mathbf{S}) = \Phi^{(0)} + \Phi^{(1)}\Phi^{(0)} + \dots + \Phi^{(K)}\Phi^{(K-1)} \dots \Phi^{(0)} \triangleq \sum_{k=0}^K \Phi^{(k:0)}$$



- Each matrix $\Phi^{(k)}$ is parametric / learnable

EdgeNet: Edge Varying Neural Networks

- ▶ EdgeNet **replaces** the **convolutional** filter by an **edge varying filter**
- ▶ For a single feature, at layer ℓ

$$\mathbf{x}_\ell = \sigma[\mathbf{H}_\ell(\mathbf{S})\mathbf{x}_{\ell-1}] = \sigma\left[\sum_{k=0}^K \Phi_\ell^{(k:0)} \mathbf{x}_{\ell-1}\right]$$

EdgeNet: Edge Varying Neural Networks

- ▶ EdgeNet **replaces** the **convolutional** filter by an **edge varying filter**
- ▶ For a single feature, at layer ℓ

$$\mathbf{x}_\ell = \sigma[\mathbf{H}_\ell(\mathbf{S})\mathbf{x}_{\ell-1}] = \sigma\left[\sum_{k=0}^K \Phi_\ell^{(k:0)} \mathbf{x}_{\ell-1}\right]$$

- ▶ For multiple features

$$\mathbf{X}_\ell = \sigma\left[\sum_{k=0}^K \Phi_\ell^{(k:0)} \mathbf{X}_{\ell-1} \mathbf{H}_{\ell k}\right]$$

where $\mathbf{H}_{\ell k} \in \mathbb{R}^{F \times F}$ is a feature mixing parameter matrix

EdgeNet Properties

- ▶ For an order- K edge varying filter
 - $\Rightarrow K + 1$ parameter matrices $\Phi^{(0)} \dots \Phi^{(K)}$ are needed with support $\mathbf{S} + \mathbf{I}$
 - $\Rightarrow K(M + N) + N$ parameters in total <- good and bad in the same time
- ▶ In EdgeNets: scale by F^2 and L

EdgeNet Properties

- ▶ For an order- K edge varying filter
 - $\Rightarrow K + 1$ parameter matrices $\Phi^{(0)} \dots \Phi^{(K)}$ are needed with support $\mathbf{S} + \mathbf{I}$
 - $\Rightarrow K(M + N) + N$ parameters in total <- good and bad in the same time
- ▶ In EdgeNets: scale by F^2 and L
- ▶ Permutation equivariance does not hold for EdgeNets
 - $\Rightarrow$ Due to the edge weight parameterization in $\Phi^{(k)}$
 - $\Rightarrow$ EdgeNet works with a fixed graph (fixed labeling / transductive)
 - $\Rightarrow$ Appropriate for: node and graph signal classification, distributed learning
 - $\Rightarrow$ Not appropriate for graph classification

Learning activity: When is permutation equivariance useful?

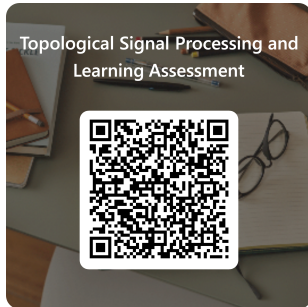
Q3.1: In a graph learning problem, when is permutation equivariance most useful?

► Instructions

- ⇒ Scan the QR code to open the activity
- ⇒ Work individually first
- ⇒ Then discuss your answer in pairs or groups of 3
- ⇒ Submit your final answer in Microsoft Forms

► Logistics

- ⇒ **Time:** 10 min
- ⇒ **Goal:** Understand when parameter sharing and permutation equivariance are beneficial
- ⇒ **Product:** One submitted response per student



<https://forms.office.com/e/yTag1Njapt>

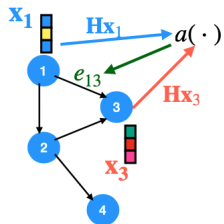
Attention Principle

Attention-based GNNs

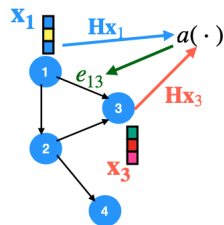
- ▶ GNNs introduced so far require the graph to be **well established**
 - ⇒ Edge weights are often **unknown** or **noisy**
- ▶ **EdgeNet**: learn the edge weights from data
 - ⇒ Too **many** parameters
- ▶ Use attention mechanism to learn the graph weights
 - ⇒ **#parameter independent** on the graph dimensions
 - ⇒ **Inductive** solution

Attention Score

- Idea: Learn **parametric** edge weights from the node features



Attention Score

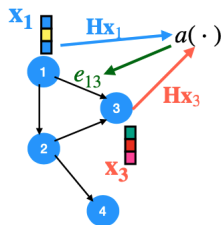


- **Idea:** Learn **parametric** edge weights from the node features
- A function $a : \mathbb{R}^F \times \mathbb{R}^F \rightarrow \mathbb{R}$ computes **attention** coefficients

$$e_{ij} = a(\mathbf{H}\mathbf{x}_i, \mathbf{H}\mathbf{x}_j)$$

- e_{ij} : **importance** of node j 's feature to node i
 - $\Rightarrow \mathbf{x}_i \in \mathbb{R}^F$ are the features of node i
 - $\Rightarrow \mathbf{H} \in \mathbb{R}^{F \times F}$ is the linear transformation matrix

Attention Score

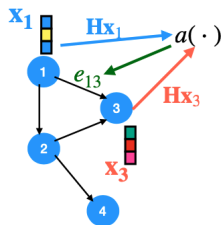


- **Idea:** Learn **parametric** edge weights from the node features
- A function $a : \mathbb{R}^F \times \mathbb{R}^F \rightarrow \mathbb{R}$ computes **attention** coefficients

$$e_{ij} = a(\mathbf{H}\mathbf{x}_i, \mathbf{H}\mathbf{x}_j)$$

- e_{ij} : **importance** of node j 's feature to node i
 - $\Rightarrow \mathbf{x}_i \in \mathbb{R}^F$ are the features of node i
 - $\Rightarrow \mathbf{H} \in \mathbb{R}^{F \times F}$ is the linear transformation matrix
- Only compute e_{ij} for neighbors $j \in \mathcal{N}_i$
 - $\Rightarrow$ Preserves the node **locality**
 - $\Rightarrow$ edge existence is the inductive bias not weight

Attention Score



- **Idea:** Learn **parametric** edge weights from the node features
- A function $a : \mathbb{R}^F \times \mathbb{R}^F \rightarrow \mathbb{R}$ computes **attention** coefficients

$$e_{ij} = a(Hx_i, Hx_j)$$

- e_{ij} : **importance** of node j 's feature to node i
 - $\Rightarrow x_i \in \mathbb{R}^F$ are the features of node i
 - $\Rightarrow H \in \mathbb{R}^{F \times F}$ is the linear transformation matrix
- Only compute e_{ij} for neighbors $j \in \mathcal{N}_i$
 - $\Rightarrow$ Preserves the node **locality**
 - $\Rightarrow$ edge existence is the inductive bias not weight
- Perform **local normalization**

$$\alpha_{ij} = \text{softmax}(e_{ij})$$

$\Rightarrow$ Induces sparsity in the parameters learned

Math Behind Attention Score

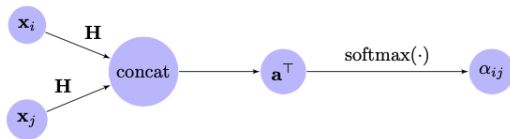
- General function $a(\cdot)$

$$e_{ij} = a(\mathbf{H}\mathbf{x}_i, \mathbf{H}\mathbf{x}_j) \xrightarrow{\text{normalization}} \alpha_{ij} = \text{softmax}(e_{ij}) = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}_i \cup i} \exp(e_{ik})}$$

Math Behind Attention Score

- General function $a(\cdot)$

$$e_{ij} = a(\mathbf{H}\mathbf{x}_i, \mathbf{H}\mathbf{x}_j) \xrightarrow{\text{normalization}} \alpha_{ij} = \text{softmax}(e_{ij}) = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}_i \cup i} \exp(e_{ik})}$$



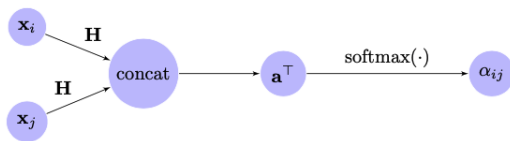
- Linear transform $\mathbf{a}^\top$

$$\alpha_{ij} = \mathbf{a}^\top [\mathbf{H}\mathbf{x}_i \parallel \mathbf{H}\mathbf{x}_j]$$

Math Behind Attention Score

- General function $a(\cdot)$

$$e_{ij} = a(\mathbf{H}\mathbf{x}_i, \mathbf{H}\mathbf{x}_j) \xrightarrow{\text{normalization}} \alpha_{ij} = \text{softmax}(e_{ij}) = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}_i \cup i} \exp(e_{ik})}$$



- Linear transform $\mathbf{a}^\top$

$$\alpha_{ij} = \mathbf{a}^\top [\mathbf{H}\mathbf{x}_i \| \mathbf{H}\mathbf{x}_j] \xrightarrow{\text{normalization}} \alpha_{ij} = \frac{\exp(\mathbf{a}^\top [\mathbf{H}\mathbf{x}_i \| \mathbf{H}\mathbf{x}_j])}{\sum_{k \in \mathcal{N}_i \cup i} \exp(\mathbf{a}^\top [\mathbf{H}\mathbf{x}_i \| \mathbf{H}\mathbf{x}_k])}$$

$\Rightarrow \|$ - concatenation operator

$\Rightarrow$ Performed for all edges $(i, j) \in \mathcal{E}$

Graph Attention Network

- ▶ We are **learning the shift operator** from features $\Phi_{ij} = \alpha_{ij}$
- ▶ Layer ℓ propagation rule

$$\mathbf{X}_\ell = \sigma[\Phi \mathbf{X}_{\ell-1} \mathbf{H}]$$

$\Rightarrow$ Order $K = 1$ GCNN on the **attention learned graph** $\mathbf{S} = \Phi$

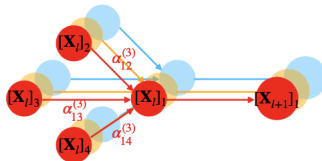
Graph Attention Network

- ▶ We are **learning the shift operator** from features $\Phi_{ij} = \alpha_{ij}$
- ▶ Layer ℓ propagation rule

$$\mathbf{X}_\ell = \sigma[\Phi \mathbf{X}_{\ell-1} \mathbf{H}]$$

⇒ Order $K = 1$ GCNN on the **attention learned graph $\mathbf{S} = \Phi$**

- ▶ Multihead Attention ⇒ Perform P GATs in parallel
- ⇒ $\Phi^{(p)}$ is the graph learned by the p th GAT



Graph Attention Network

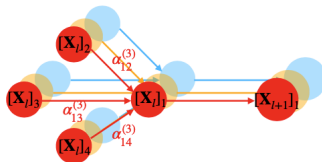
- ▶ We are **learning the shift operator** from features $\Phi_{ij} = \alpha_{ij}$
- ▶ Layer ℓ propagation rule

$$\mathbf{X}_\ell = \sigma[\Phi \mathbf{X}_{\ell-1} \mathbf{H}]$$

$\Rightarrow$ Order $K = 1$ GCNN on the **attention learned graph $\mathbf{S} = \Phi$**

- ▶ Multihead Attention $\Rightarrow$ Perform P GATs in parallel

$\Rightarrow \Phi^{(p)}$ is the graph learned by the p th GAT



$\Rightarrow$ Concatenate the P outputs $\parallel_p: \mathbf{X}_\ell = \parallel_{p=1}^P \sigma[\Phi^{(p)} \mathbf{X}_{\ell-1} \mathbf{H}^{(p)}]$

Graph Attention Network

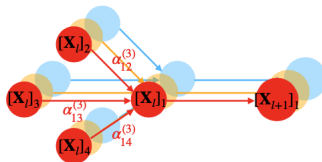
- ▶ We are **learning the shift operator** from features $\Phi_{ij} = \alpha_{ij}$
- ▶ Layer ℓ propagation rule

$$\mathbf{X}_\ell = \sigma[\Phi \mathbf{X}_{\ell-1} \mathbf{H}]$$

$\Rightarrow$ Order $K = 1$ GCNN on the **attention learned graph** $\mathbf{S} = \Phi$

- ▶ Multihead Attention $\Rightarrow$ Perform P GATs in parallel

$\Rightarrow \Phi^{(p)}$ is the graph learned by the p th GAT



$\Rightarrow$ Concatenate the P outputs $\parallel_p: \mathbf{X}_\ell = \parallel_{p=1}^P \sigma[\Phi^{(p)} \mathbf{X}_{\ell-1} \mathbf{H}^{(p)}]$

$\Rightarrow$ Average final layer L across P : $\mathbf{X}_L = \left[\frac{1}{P} \sum_{p=1}^P \sigma[\Phi^{(p)} \mathbf{X}_{L-1} \mathbf{H}^{(p)}] \right]$

GAT Properties

- ▶ Attention coefficient for nodes i and j are different: $\alpha_{ij} \neq \alpha_{ji}$

GAT Properties

- ▶ Attention coefficient for nodes i and j are different: $\alpha_{ij} \neq \alpha_{ji}$
- ▶ Parameters (**independent** of graph dimensions) – $\alpha_{ij} = \sigma[\mathbf{a}^\top [\mathbf{H}\mathbf{x}_i || \mathbf{H}\mathbf{x}_j]]$
 - $\Rightarrow$ #parameters per layer is $F^2 + 2F$
 - $\Rightarrow F^2$ parameters in $\mathbf{H}$ and $2F$ in $\mathbf{a}$
- ▶ **Inductive architecture**

GAT Properties

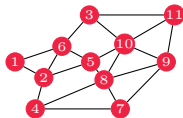
- ▶ Attention coefficient for nodes i and j are different: $\alpha_{ij} \neq \alpha_{ji}$
- ▶ Parameters (**independent** of graph dimensions) – $\alpha_{ij} = \sigma[\mathbf{a}^\top [\mathbf{H}\mathbf{x}_i || \mathbf{H}\mathbf{x}_j]]$
 - $\Rightarrow$ #parameters per layer is $F^2 + 2F$
 - $\Rightarrow F^2$ parameters in $\mathbf{H}$ and $2F$ in $\mathbf{a}$
- ▶ **Inductive architecture**

Q: When to use attention?

Message Passing Principle

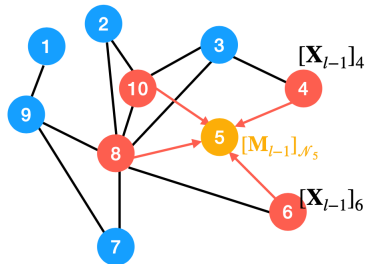
Message Passing Neural Networks

- ▶ **GNNs so far:** composition of graph filters and nonlinearity
- ▶ GNNs can also be motivated through neural message passing
- ▶ **Main Idea:** At layer ℓ , each node i
 - ⇒ Aggregates messages (**feature combinations**) from neighbours
 - ⇒ Updates **own** features locally



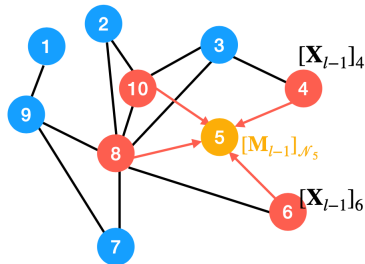
- ▶ **Examples:**
 - ⇒ GCNN for $K = 1$ - each layer is one shift;
 - ⇒ GCN
 - ⇒ GAT

Message Passing Neural Networks



- Node 5 $\Rightarrow$ Current state $[\mathbf{X}_{l-1}]_5$
 - $\Rightarrow \mathcal{N}_5 = \{4, 6, 8, 10\}$ pass information to 5
 - $\Rightarrow$ Node 5 aggregates them to form message $[\mathbf{M}_{l-1}]_{\mathcal{N}_5}$

Message Passing Neural Networks



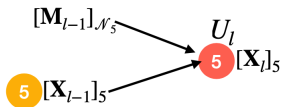
- Node 5 $\Rightarrow$ Current state $[\mathbf{X}_{l-1}]_5$
 - $\Rightarrow \mathcal{N}_5 = \{4, 6, 8, 10\}$ pass information to 5
 - $\Rightarrow$ Node 5 aggregates them to form message $[\mathbf{M}_{l-1}]_{\mathcal{N}_5}$

- Aggregate messages:

$$[\mathbf{M}_{l-1}]_{\mathcal{N}_i} = \mathbf{A}_\ell(\{[\mathbf{X}_{l-1}]_j, \forall j \in \mathcal{N}_i\})$$

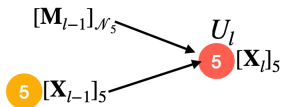
- $\mathbf{A}_\ell(\cdot)$ is an aggregate function, which can be neural networks, linear transforms

Message Passing Neural Networks



- State $[\mathbf{X}_{\ell-1}]_5$, Messages $[\mathbf{M}_{\ell-1}]_{\mathcal{N}_5}$
⇒ Note 5 updates features $[\mathbf{X}_{\ell}]_5$

Message Passing Neural Networks



- State $[\mathbf{X}_{\ell-1}]_5$, Messages $[\mathbf{M}_{\ell-1}]_{\mathcal{N}_5}$
⇒ Note 5 updates features $[\mathbf{X}_\ell]_5$

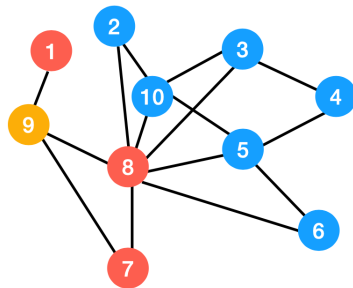
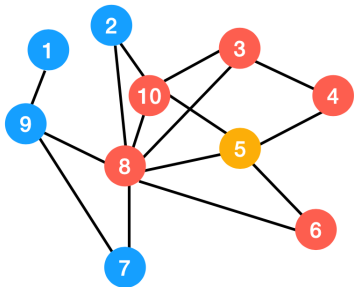
- Update features :

$$[\mathbf{X}_\ell]_i = U_\ell([\mathbf{X}_{\ell-1}]_i, [\mathbf{M}_{\ell-1}]_{\mathcal{N}_i})$$

- $U_\ell(\cdot)$ is the update function, which can be neural networks, linear transforms

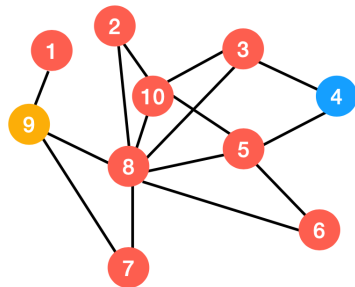
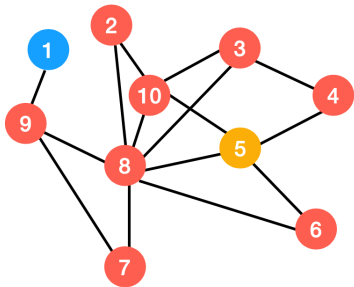
Message Passing Neural Networks

- ▶ **Neighborhood reach:**
 - ⇒ After one message passing / one layer
 - ⇒ Each node contains information of its one-hop neighbours
- ▶ Node 5 and Node 9 reach their one-hop neighborhood



Message Passing Neural Networks

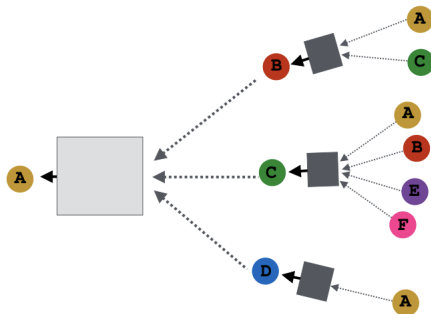
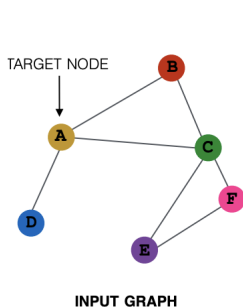
- ▶ **Neighborhood reach:**
 - ⇒ After K message passing / one layer
 - ⇒ Each node contains information of its K -hop neighbours
- ▶ Node 5 and Node 9 reach their two-hop neighborhood



MPNN Computation Graph

► Graph Size:

- ⇒ As the graph grows larger, so does the computational and memory requirements of the network.
- ⇒ Multiple iterations of message passing lead to linearly increasing computational costs
- ⇒ Storing the computation graph in memory becomes challenging as the graph size increases.



MPNN Examples

- ▶ **Building a MPNN** $\Rightarrow$ **instantiate** the aggregate and update functions
 - $\Rightarrow$ Could be arbitrary complex functions

MPNN Examples

- ▶ **Building a MPNN** $\Rightarrow$ **instantiate** the aggregate and update functions
 - $\Rightarrow$ Could be arbitrary complex functions
- ▶ **Example:**
 - $\Rightarrow$ **Aggregate** $\rightarrow [\mathbf{M}_{\ell-1}]_{\mathcal{N}_i} = \sum_{j \in \mathcal{N}_i} [\mathbf{X}_{\ell-1}]_j$
 - $\Rightarrow$ **Update** $\rightarrow$ linear transform + pointwise nonlinearity GCN

$$[\mathbf{X}_{\ell}]_i = \sigma [\mathbf{H}_{0\ell} [\mathbf{X}_{\ell-1}]_i + \mathbf{H}_{1\ell} [\mathbf{M}_{\ell-1}]_{\mathcal{N}_i}]$$

$$\mathbf{X}_{\ell} = \sigma [\mathbf{X}_{\ell-1} \mathbf{H}_{0\ell} + \mathbf{S} \mathbf{X}_{\ell-1} \mathbf{H}_{1\ell}]$$

MPNN Examples

- ▶ **Building a MPNN** $\Rightarrow$ **instantiate** the aggregate and update functions
 $\Rightarrow$ Could be arbitrary complex functions

- ▶ **Example:**

$\Rightarrow$ **Aggregate** $\rightarrow [\mathbf{M}_{\ell-1}]_{\mathcal{N}_i} = \sum_{j \in \mathcal{N}_i} [\mathbf{X}_{\ell-1}]_j$

$\Rightarrow$ **Update** $\rightarrow$ linear transform + pointwise nonlinearity GCN

$$[\mathbf{X}_{\ell}]_i = \sigma [\mathbf{H}_{0\ell}[\mathbf{X}_{\ell-1}]_i + \mathbf{H}_{1\ell}[\mathbf{M}_{\ell-1}]_{\mathcal{N}_i}]$$

$$\mathbf{X}_{\ell} = \sigma [\mathbf{X}_{\ell-1} \mathbf{H}_{0\ell} + \mathbf{S} \mathbf{X}_{\ell-1} \mathbf{H}_{1\ell}]$$

- ▶ **Example GIN:** Graph Isomorphism Network

$$[\mathbf{X}_{\ell}]_i = \text{MLP}_{\ell} \left((1 + \epsilon_{\ell}) [\mathbf{X}_{(\ell-1)}]_i + \sum_{j \in \mathcal{N}_i} [\mathbf{X}_{(\ell-1)}]_j \right)$$

$\Rightarrow$ where ϵ_{ℓ} is a learnable/fixed scalar

MPNN Examples

- ▶ **Building a MPNN** $\Rightarrow$ **instantiate** the aggregate and update functions
 $\Rightarrow$ Could be arbitrary complex functions

- ▶ **Example:**

$\Rightarrow$ **Aggregate** $\rightarrow [\mathbf{M}_{\ell-1}]_{\mathcal{N}_i} = \sum_{j \in \mathcal{N}_i} [\mathbf{X}_{\ell-1}]_j$

$\Rightarrow$ **Update** $\rightarrow$ linear transform + pointwise nonlinearity GCN

$$[\mathbf{X}_{\ell}]_i = \sigma [\mathbf{H}_{0\ell} [\mathbf{X}_{\ell-1}]_i + \mathbf{H}_{1\ell} [\mathbf{M}_{\ell-1}]_{\mathcal{N}_i}]$$

$$\mathbf{X}_{\ell} = \sigma [\mathbf{X}_{\ell-1} \mathbf{H}_{0\ell} + \mathbf{S} \mathbf{X}_{\ell-1} \mathbf{H}_{1\ell}]$$

- ▶ **Example GIN:** Graph Isomorphism Network

$$[\mathbf{X}_{\ell}]_i = \text{MLP}_{\ell} \left((1 + \epsilon_{\ell}) [\mathbf{X}_{(\ell-1)}]_i + \sum_{j \in \mathcal{N}_i} [\mathbf{X}_{(\ell-1)}]_j \right)$$

$\Rightarrow$ where ϵ_{ℓ} is a learnable/fixed scalar

- ▶ **Q:** What are the dimensions of the MLP matrices?

MPNN Properties

- ▶ Simple and efficient implementation **one-hop operation per layer**
- ▶ Permutation equivariant
- ▶ **Other** **aggregate** and **update** functions lead to **other GNNs**
 - ⇒ E.g., use MLP to transform features
 - ⇒ Can learn also from edge weights ⇒ put them in the function

Learning activity: Choosing a GNN architecture

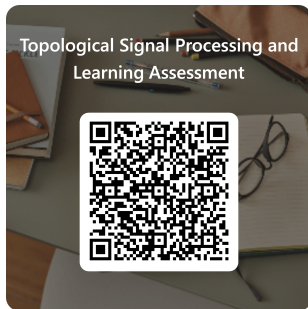
Q3.2: A researcher wants to perform node classification on a fixed graph. The model should allow different neighbors to contribute differently to each node, and permutation equivariance is not essential. Which architecture is the best fit?

► Instructions

- ⇒ Scan the QR code to open the activity
- ⇒ Work individually first
- ⇒ Then discuss your answer in pairs or groups of 3
- ⇒ Submit your final answer in Microsoft Forms

► Logistics

- ⇒ **Time:** 10 min
- ⇒ **Goal:** Distinguish when edge-varying parameters are more suitable than shared-parameter or attention-based graph convolutions
- ⇒ **Product:** One submitted response per student



<https://forms.office.com/e/yTag1Njapt>

- ▶ (Isufi et al) Graph filters for processing and learning from network data
⇒ Ch. 5, 6, 7
- ▶ (Isufi et al) Graph filters for signal processing and machine learning on graphs
⇒ Sec. VI. Sec. VIII
- ▶ (Wu et al) A comprehensive survey on graph neural networks
⇒ Excluding VI, VII
- ▶ (Isufi et al) EdgeNets: Edge Varying Graph Neural Networks
⇒ Excluding Sec. 4.2; 4.3
- ▶ Ma and Tang, Deep Learning on Graphs https://web.njit.edu/~ym329/dlg_book/
⇒ Sec.: 5.1, 5.2, 5.4, 5.5,

► Convolutional-based architectures

- ⇒ (ChebNet) Defferrard, M. Bresson, and Vandergheynst, P. (2016). Convolutional Neural Networks on Graphs with Fast Localized Spectral Filtering
- ⇒ (Bernet) He, M., Wei, Z., and Xu, H. (2021). Bernnet: Learning arbitrary graph spectral filters via Bernstein approximation.
- ⇒ (ChebNetII) He, et al (2022). Convolutional neural networks on graphs via Chebyshev approximation, revisited
- ⇒ (Jacobinet) Wang and Zhan (2022). How powerful are spectral graph neural networks

► Attention-based architectures

- ⇒ (GAT) Velickovic et al. "Graph attention networks" ICLR, 2017
- ⇒ (GATv2) Brody et al. "How Attentive Are Graph Attention Networks "ICLR, 2022

► Message passing architectures

- ⇒ Gilmer et al. "Neural Message Passing for Quantum Chemistry" ICML, 2017
- ⇒ Xu et al. "How powerful are graph neural networks" ICLR 2019
- ⇒ Hamilton et al. "Inductive representation learning on large graphs", NeurIPS, 2018

Further reading

- Pooling

- ⇒ Liu et al., Graph Pooling for Graph Neural Networks: Progress, Challenges, and Opportunities

- Full GNN pipeline

- ⇒ You et al., Design space for graph neural networks, NeurIPS, 2020

- Theoretical insights

- ⇒ (Inductive bias) Battaglia et al., Relational inductive biases, deep learning and graph networks, 2018

Video material

- ▶ Graph Machine Learning Course – Stanford: <https://web.stanford.edu/class/cs224w/>
⇒ Lectures 3, 4, 5
- ▶ Graph Neural Networks – UPENN: <https://www.youtube.com/@alelabalelab5337/playlists>
⇒ Lectures: 4, 5

Backup Slides

Approximate Personalized Propagation of Neural Prediction

- **Idea:** PageRank works – reach nodes in that way
 - ⇒ $\pi_{\text{pr}} \in [0, 1]^N$: probability vector of being at each node
 - ⇒ PageRank updates (shifts) the probabilities as

$$\pi_{\text{pr}} = \mathbf{A}_{\text{rw}} \pi_{\text{pr}} = \mathbf{A} \mathbf{D}^{-1} \pi_{\text{pr}}$$

Approximate Personalized Propagation of Neural Prediction

- **Idea:** PageRank works – reach nodes in that way
 - ⇒ $\pi_{\text{pr}} \in [0, 1]^N$: probability vector of being at each node
 - ⇒ PageRank updates (shifts) the probabilities as

$$\pi_{\text{pr}} = \mathbf{A}_{\text{rw}} \pi_{\text{pr}} = \mathbf{A} \mathbf{D}^{-1} \pi_{\text{pr}}$$

- **Personalize PageRank:** consider a root node n with the one-hot indicator vector $\mathbf{i}_n$
 - ⇒ Shift the probabilities as

$$\pi_{\text{ppr}}(\mathbf{i}_n) = (1 - h) \mathbf{A}_{\text{rw}} \pi_{\text{ppr}}(\mathbf{i}_n) + h \mathbf{i}_n$$

with the teleport (or restart) probability $h \in [0, 1]$

Approximate Personalized Propagation of Neural Prediction

- **Idea:** PageRank works – reach nodes in that way
 - ⇒ $\pi_{\text{pr}} \in [0, 1]^N$: probability vector of being at each node
 - ⇒ PageRank updates (shifts) the probabilities as

$$\pi_{\text{pr}} = \mathbf{A}_{\text{rw}} \pi_{\text{pr}} = \mathbf{A} \mathbf{D}^{-1} \pi_{\text{pr}}$$

- **Personalize PageRank:** consider a root node n with the one-hot indicator vector $\mathbf{i}_n$
 - ⇒ Shift the probabilities as

$$\pi_{\text{ppr}}(\mathbf{i}_n) = (1 - h) \mathbf{A}_{\text{rw}} \pi_{\text{ppr}}(\mathbf{i}_n) + h \mathbf{i}_n$$

with the teleport (or restart) probability $h \in [0, 1]$

- **Solving the PPR** we get

$$\pi_{\text{ppr}}(\mathbf{i}_n) = h(\mathbf{I} - (1 - h) \mathbf{A}_{\text{rw}})^{-1} \mathbf{i}_n$$

Approximate Personalized Propagation of Neural Prediction

- ▶ **Idea:** PageRank works – reach nodes in that way
 - ⇒ $\pi_{\text{pr}} \in [0, 1]^N$: probability vector of being at each node
 - ⇒ PageRank updates (shifts) the probabilities as

$$\pi_{\text{pr}} = \mathbf{A}_{\text{rw}} \pi_{\text{pr}} = \mathbf{A} \mathbf{D}^{-1} \pi_{\text{pr}}$$

- ▶ **Personalize PageRank:** consider a root node n with the one-hot indicator vector $\mathbf{i}_n$
 - ⇒ Shift the probabilities as

$$\pi_{\text{ppr}}(\mathbf{i}_n) = (1 - h) \mathbf{A}_{\text{rw}} \pi_{\text{ppr}}(\mathbf{i}_n) + h \mathbf{i}_n$$

with the teleport (or restart) probability $h \in [0, 1]$

- ▶ **Solving the PPR** we get

$$\pi_{\text{ppr}}(\mathbf{i}_n) = h(\mathbf{I} - (1 - h) \mathbf{A}_{\text{rw}})^{-1} \mathbf{i}_n$$

- ▶ The teleport vector $\mathbf{i}_n$ preserves the node's local neighborhood **Q:** Why?

- Thus, we have an **inverse filtering matrix**

$$\boldsymbol{\pi}_{\text{ppr}}(\mathbf{i}_n) = h(\mathbf{I} - (1 - h)\mathbf{A}_{\text{rw}})^{-1}\mathbf{i}_n \quad \mathbf{H}(\mathbf{S}) := h(\mathbf{I}_n - (1 - h)\mathbf{A}_{\text{rw}})^{-1}$$

- Similar to Tikhonov regularization (Lec. 2)
 $\Rightarrow$ **Q**: What does this mean?

- Thus, we have an **inverse filtering matrix**

$$\boldsymbol{\pi}_{\text{ppr}}(\mathbf{i}_n) = h(\mathbf{I} - (1 - h)\mathbf{A}_{\text{rw}})^{-1}\mathbf{i}_n \quad \mathbf{H}(\mathbf{S}) := h(\mathbf{I}_n - (1 - h)\mathbf{A}_{\text{rw}})^{-1}$$

- Similar to Tikhonov regularization (Lec. 2)

⇒ **Q**: What does this mean?

- Thus, we can build a GNN as per

$$\mathbf{Z}_{\text{PPNP}} = \text{softmax}(h(\mathbf{I} - (1 - h)\mathbf{A}_{\text{rw}})^{-1}\mathbf{X}') \quad \text{with} \quad \mathbf{X}' = f_{\boldsymbol{\theta}}(\mathbf{X})$$

⇒ $\mathbf{X}$ is the feature matrix and $f_{\boldsymbol{\theta}}$ is a feature transformation function (e.g., multi-layer perceptron) with parameters $\boldsymbol{\theta}$ generating the transformed features $\mathbf{X}'$.

- ▶ Thus, we have an **inverse filtering matrix**

$$\pi_{\text{ppr}}(\mathbf{i}_n) = h(\mathbf{I} - (1 - h)\mathbf{A}_{\text{rw}})^{-1}\mathbf{i}_n \quad \mathbf{H}(\mathbf{S}) := h(\mathbf{I}_n - (1 - h)\mathbf{A}_{\text{rw}})^{-1}$$

- ▶ Similar to Tikhonov regularization (Lec. 2)

⇒ **Q**: What does this mean?

- ▶ Thus, we can build a GNN as per

$$\mathbf{Z}_{\text{PPNP}} = \text{softmax}(h(\mathbf{I} - (1 - h)\mathbf{A}_{\text{rw}})^{-1}\mathbf{X}') \quad \text{with} \quad \mathbf{X}' = f_{\theta}(\mathbf{X})$$

⇒ $\mathbf{X}$ is the feature matrix and f_{θ} is a feature transformation function (e.g., multi-layer perceptron) with parameters θ generating the transformed features $\mathbf{X}'$.

- ▶ Computing the inverse is challenging

⇒ **Cubic complexity**

⇒ **Objective**: approximate the inverse

- ▶ Matrix inverse identity: $h(\mathbf{I} + h\mathbf{A})^{-1} = h \sum_{k=0}^{\infty} (-1)^k h^k \mathbf{A}^k$
- ▶ PPNP becomes

$$\mathbf{Z}_{\text{PPNP}} = \text{softmax} \left(h \sum_{k=0}^{\infty} (-1)^k (h-1)^k \mathbf{A}_{\text{rw}}^k \mathbf{X}' \right)$$

- $\Rightarrow$ A particular GCNN of one layer with order $K = \infty$
- $\Rightarrow$ **Truncating** $K = L$ leads to the APPNP of L layers

- ▶ Matrix inverse identity: $h(\mathbf{I} + h\mathbf{A})^{-1} = h \sum_{k=0}^{\infty} (-1)^k h^k \mathbf{A}^k$

- ▶ PPNP becomes

$$\mathbf{Z}_{\text{PPNP}} = \text{softmax} \left(h \sum_{k=0}^{\infty} (-1)^k (h-1)^k \mathbf{A}_{\text{rw}}^k \mathbf{X}' \right)$$

$\Rightarrow$ A particular GCNN of one layer with order $K = \infty$

$\Rightarrow$ **Truncating** $K = L$ leads to the APPNP of L layers

- ▶ **APPNP** can also be implemented as

$$\mathbf{X}_0 = \mathbf{X}' = f_{\theta}(\mathbf{X})$$

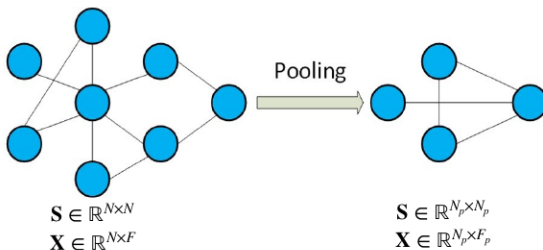
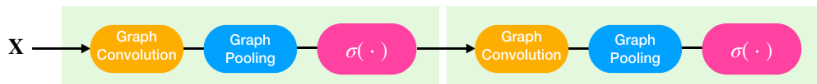
$$\mathbf{X}_{\ell+1} = (1-h)\mathbf{A}_{\text{rw}}\mathbf{X}_{\ell} + h\mathbf{X}'$$

$$\mathbf{X}_L = \text{softmax}((1-h)\mathbf{A}_{\text{rw}}\mathbf{X}_{L-1} + h\mathbf{X}')$$

Graph Pooling

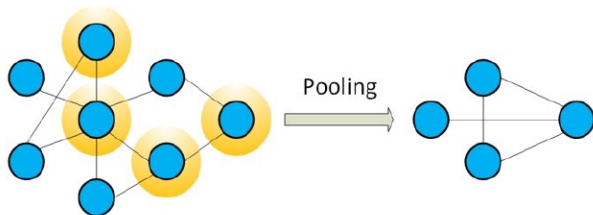
Graph Pooling

- Reduce the graph between convolutional (or other) layers.



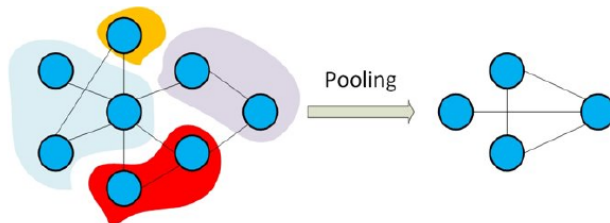
- Not central for most GNNs
- Lots of techniques
 - ⇒ Node selection
 - ⇒ Clustering
 - ⇒ Zero-padding
 - ⇒ Global aggregation

Node Selection



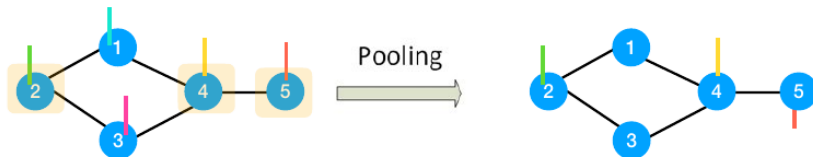
- A few important nodes are kept
 - ⇒ Define a metric for sampling (random walk, node centrality, degree)
 - ⇒ Edges ...
 - ⇒ Features ...
 - ⇒ Fast implementation

Clustering



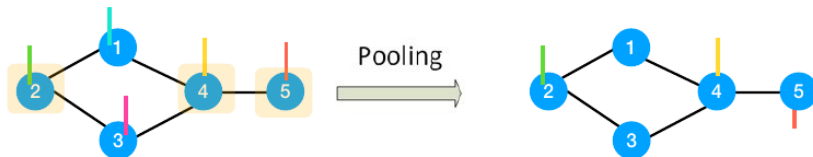
- Collect nodes in a cluster
 - ⇒ Now build **super nodes**
 - ⇒ **Edges** connect clusters
 - ⇒ **Features**: mean or max values within cluster
 - ⇒ Not implementation friendly

Zero-pad pooling



- Do **not** change the graph - put zero to not important nodes
 - ⇒ 1- **Node Downsample**: select a few active nodes (centrality, degree)
 - ⇒ 2- **Feature summarization**: mean or max in a neighborhood
- Sparsity in features - fast implementation

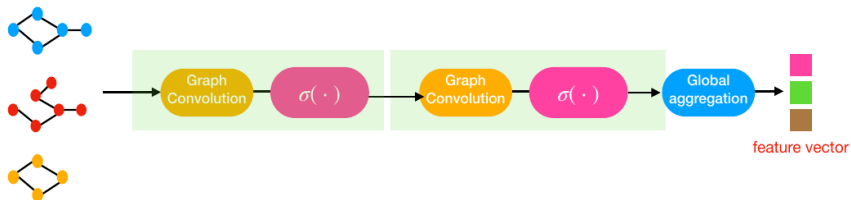
Zero-pad pooling



- Do **not** change the graph - put zero to not important nodes
 - ⇒ 1- **Node Downsample**: select a few active nodes (centrality, degree)
 - ⇒ 2- **Feature summarization**: mean or max in a neighborhood
- Sparsity in features - fast implementation

Q: When we should not change the graph?

Global Aggregation



- ▶ Useful for graph classification tasks
- ▶ Aggregate all node feature in a global vector of features
⇒ mean, max, sum
- ▶ Accepts graphs of different sizes
⇒ Useful for graph classification

Issues with Global Aggregation

- ▶ Global aggregation will destroy the learned embeddings
 - ⇒ Intuition: because we leverage the graph structure till there and after we ignore it

Issues with Global Aggregation

- ▶ **Global aggregation** will destroy the learned embeddings
 - ⇒ **Intuition**: because we leverage the graph structure till there and after we **ignore** it
- ▶ **Example**: Consider 1-dimensional embeddings
 - ⇒ **Graph $\mathcal{G}_1$** : output layer per node $\mathbf{x}_L^1 = [-1, -2, 0.1, 2]^\top$
 - ⇒ **Graph $\mathcal{G}_2$** : output layer per node $\mathbf{x}_L^2 = [0, 0, 0.0, 0]^\top$

Issues with Global Aggregation

- ▶ **Global aggregation** will destroy the learned embeddings
 - ⇒ **Intuition**: because we leverage the graph structure till there and after we **ignore** it
- ▶ **Example**: Consider 1-dimensional embeddings
 - ⇒ **Graph $\mathcal{G}_1$** : output layer per node $\mathbf{x}_L^1 = [-1, -2, 0.1, 2]^\top$
 - ⇒ **Graph $\mathcal{G}_2$** : output layer per node $\mathbf{x}_L^2 = [0, 0, 0.0, 0]^\top$
- ▶ **Graph $\mathcal{G}_1$** and **Graph $\mathcal{G}_2$** lead to different node embeddings
 - ⇒ Should suffice to distinguish the graphs

Issues with Global Aggregation

- ▶ **Global aggregation** will destroy the learned embeddings
 - ⇒ **Intuition**: because we leverage the graph structure till there and after we **ignore** it
- ▶ **Example**: Consider 1-dimensional embeddings
 - ⇒ **Graph $\mathcal{G}_1$** : output layer per node $\mathbf{x}_L^1 = [-1, -2, 0.1, 2]^\top$
 - ⇒ **Graph $\mathcal{G}_2$** : output layer per node $\mathbf{x}_L^2 = [0, 0, 0.0, 0]^\top$
- ▶ **Graph $\mathcal{G}_1$** and **Graph $\mathcal{G}_2$** lead to different node embeddings
 - ⇒ Should suffice to distinguish the graphs
- ▶ **Global mean** pooling
 - ⇒ **Graph $\mathcal{G}_1$** $\hat{y}_{\mathcal{G}_1} = \text{mean}\{1, -2, 0.1, 2\} = 0$
 - ⇒ **Graph $\mathcal{G}_2$** $\hat{y}_{\mathcal{G}_2} = \text{mean}\{0, 0, 0.0, 0\} = 0$

Issues with Global Aggregation

- ▶ **Global aggregation** will destroy the learned embeddings
 - ⇒ **Intuition**: because we leverage the graph structure till there and after we **ignore** it
- ▶ **Example**: Consider 1-dimensional embeddings
 - ⇒ **Graph $\mathcal{G}_1$** : output layer per node $\mathbf{x}_L^1 = [-1, -2, 0.1, 2]^\top$
 - ⇒ **Graph $\mathcal{G}_2$** : output layer per node $\mathbf{x}_L^2 = [0, 0, 0.0, 0]^\top$
- ▶ **Graph $\mathcal{G}_1$** and **Graph $\mathcal{G}_2$** lead to different node embeddings
 - ⇒ Should suffice to distinguish the graphs
- ▶ **Global mean pooling**
 - ⇒ **Graph $\mathcal{G}_1$** $\hat{y}_{\mathcal{G}_1} = \text{mean}\{1, -2, 0.1, 2\} = 0$
 - ⇒ **Graph $\mathcal{G}_2$** $\hat{y}_{\mathcal{G}_2} = \text{mean}\{0, 0, 0.0, 0\} = 0$
 - ⇒ We **cannot** differentiate the two graphs

Issues with Global Aggregation

- Similar issues appear also for other global aggregators
 - ⇒ **Partial solution:** hierarchical pooling –reduction + global aggregation in clusters

